VIETNAM NATIONAL UNIVERSITY, HANOI

**INTERNATIONAL SCHOOL**

---

**Midterm Machine Learning**
**Report Analysis**

**Group 3**

# Currency Exchange Rate Prediction (USD/VND)

**Teacher: Trần Thị Oanh**


**Members:**

1. **Nguyễn Phạm Quỳnh Trang**
2. **Phạm Hiếu Ngân**
3. **Nguyễn Thị Hồng Nhung**

# TABLE OF CONTENTS

**1. Introduction....3**
1.1 Business Problem:....3
1.2 Benefits and Challenges of Using ML to the problem:....3
**2. Data Description....4**
**3. Machine Learning Methods....5**
3.1 Support Vector Regression (SVR)....5
3.2 Artificial Neural Networks (ANN)....7
3.2.1. Base Model....8
3.2.2. Training model....8
3.2.3. Analysis....9
3.2.4. Improve with EarlyStopping....10
3.3 Long Short-Term Memory (LSTM)....11
3.3.1. Base Model....11
3.3.2 Analysis....12
3.3.3 Improve with Early Stopping....14
**4. Fine-Tuning the Models....15**
4.1 SVR....15
4.2 ANN....16
4.3 LSTM....17
**5. Comparison and Results....18**
**6. Conclusion....19**

## 1. Introduction

### 1.1 Business Problem:

The exchange rate prediction of USD/VND plays a significant role in financial markets and businesses that deal with international transactions. Accurate predictions can help reduce risks and optimize trading strategies. However, predicting exchange rates is a challenging task due to their volatility and dependency on numerous economic factors.

The prediction of the USD/VND exchange rate plays a critical role in the financial sector, particularly for businesses and investors involved in international trade, investment, and currency markets. Accurate forecasting of exchange rate movements allows companies to manage and hedge currency risks more effectively, optimize cross-border pricing strategies, and improve financial planning. For investors and traders, reliable exchange rate predictions can enhance decision-making processes, increase portfolio returns, and mitigate losses resulting from unfavorable currency fluctuations.

However, predicting exchange rates remains an inherently complex and challenging task. The foreign exchange market is influenced by a multitude of interrelated factors, including but not limited to interest rates, inflation rates, geopolitical events, monetary policies, and market sentiment. In the specific case of the USD/VND pair, additional challenges arise from Vietnam's emerging market dynamics, regulatory interventions, and fluctuations in global commodity prices, which all contribute to heightened volatility and unpredictability.

### 1.2 Benefits and Challenges of Using ML to the problem:

The application of machine learning (ML) techniques to the problem of exchange rate prediction offers several significant advantages:

Benefits:

- Automation and efficiency: Machine learning models can automate the process of analyzing large volumes of historical data and generating predictions, making the

forecasting process faster, more scalable, and less prone to manual errors compared to traditional statistical methods.

- Enhanced accuracy: By capturing complex, nonlinear relationships within the data, machine learning algorithms often provide more accurate forecasts than classical models, especially in volatile and dynamic markets like foreign exchange.
- Adaptability: Advanced machine learning models, such as ensemble methods or deep learning networks, can adapt to changing patterns in the data over time, offering a level of flexibility that traditional econometric models might lack.

Challenges:

- Data quality and noise: Currency exchange rate data are inherently noisy, influenced by a wide range of unpredictable factors such as political events, market sentiment, and sudden economic policy shifts. Ensuring the dataset is clean, relevant, and representative is a persistent challenge in building robust models.
- Model overfitting: Machine learning models, especially those with high complexity, are at risk of overfitting to historical patterns. An overfitted model may perform exceptionally well on past data but fail to generalize to new data, thus providing unreliable real-time predictions.
- Computational resources: Training complex models, especially on large datasets or with frequent retraining to capture the latest trends, may require substantial computational power and technical infrastructure.

**2. Data Description**

The dataset used for this project provides a detailed record of historical exchange rates between the US Dollar (USD) and the Vietnamese Dong (VND) over a specific period. In addition to the raw closing exchange rates, the dataset includes a set of engineered features such as lagged values, moving averages, and standard deviations. These features are designed to support deeper statistical analysis, trend detection, and predictive modeling.

The variables included in the dataset are:

- Date: The trading date is recorded in the format yyyy-mm-dd.
- Close: The closing exchange rate of USD to VND on the given date.
- ISO Code From: The ISO 4217 currency code of the base currency, always "USD".

- ISO Code To: The ISO 4217 currency code of the quote currency, always "VND".
- Lag_1: The closing exchange rate from the previous trading day prior.
- Lag_2: The closing exchange rate from two trading days prior.
- MA_5: The 5-day moving average of the closing exchange rate.
- STD_5: The 5-day standard deviation of the closing exchange rate.
- MA_20: The 20-day moving average of the closing exchange rate.
- STD_20: The 20-day standard deviation of the closing exchange rate.

Before modeling, the dataset underwent basic preprocessing steps to ensure data quality and readiness for analysis:

- Handling missing values: The dataset was checked for missing or null values.
- Data type conversion: The "date" column was converted into a datetime format to facilitate time-based operations such as sorting and lag feature generation.
- Sorting: Records were sorted in chronological order based on the "date" column to maintain the sequential integrity of the time series.
- Feature engineering: Several lagged features ("Lag_1", "Lag_2"), moving averages ("MA_5", "MA_20"), and standard deviations ("STD_5", "STD_20") were pre-calculated and included in the dataset to capture short-term and long-term trends and volatility.

**3. Machine Learning Methods**

This project will describe the three machine learning methods you used for predicting the USD/VND exchange rate, perform fine-tuning for each, and evaluate their performance.

**3.1 Support Vector Regression (SVR)**

Support vector regression (SVR) is a machine learning algorithm built on the foundation of support vector machine (SVM), adapted specifically for regression tasks. Instead of predicting class labels, SVR is used to predict continuous values such as stock prices, exchange rates, or other time series data.

SVR works by finding a function that deviates from the actual target values by no more than a specified margin (ε), while also being as flat as possible. This allows it to tolerate a certain amount of error, defined by the epsilon value, making it robust to "noise" in the data.

Hyperparameters to Tune:

1. Regularization parameter (C): This parameter controls the trade-off between model complexity and the penalty for deviations greater than epsilon (ε).

    - Larger C makes the model attempt to fit the training data as closely as possible, which may lead to overfitting if the data is noisy.
    - Smaller C allows more flexibility, potentially avoiding overfitting, but could result in underfitting if the model does not capture important patterns.
2. Epsilon value: determines how much error is acceptable when making predictions. It defines a tolerance margin around the predicted values where small errors are ignored.

    - Smaller epsilon: meaning the model is stricter about small errors and tries to fit the data more precisely. This can result in more accuracy, but it might cause the model to overfit the training data.
    - Larger epsilon: allowing more errors without penalizing them. This creates a simpler model with fewer support vectors, but it could lead to the model underfitting the data.

3. Kernel type: determines how the model transforms the input data into a higher-dimensional space, making it easier to learn relationships between features and the target variable.

    The radial basis function kernel (RBF) is one of the most commonly used kernels and is particularly well-suited for non-linear data. It works by mapping data into an infinite-dimensional space, where it calculates the distance between each data point and other points. This transformation leads to more flexible and complex decision boundaries, allowing the model to capture intricate relationships in the data. The RBF kernel is ideal when the relationship between the features and the target is non-linear, as it helps the model handle complex, non-linear patterns effectively.

4. Gamma: is a key parameter in the RBF kernel, determining how much influence a single training example has on the model's predictions. Specifically, it controls how far the effect of a data point "spreads" in the feature space.

- A small gamma will allow each data point to have a wider influence, resulting in a smoother decision boundary and a model that is less sensitive to specific data points.
- A large gamma makes each data point influence a smaller region of the feature space, enabling the model to capture more detailed relationships but also risking overfitting.

**3.2 Artificial Neural Networks (ANN)**

Model Overview:

An **Artificial Neural Network (ANN)** is a computational model inspired by the human brain, consisting of interconnected layers of neurons (input, hidden, output layers) used to capture complex patterns in data and make predictions or classifications. ANN learns patterns by adjusting weights to reduce prediction errors.

ANNs mimic the way the human brain works and are capable of capturing complex relationships in data.

1. Number of hidden layers
   Controls the depth (complexity) of the ANN—more layers can model more complex patterns but risk overfitting.
2. Number of neurons per layer
   Determines the network's capacity to learn. More neurons capture finer details but may overfit.
3. Activation function (ReLU, Tanh, Sigmoid…)
   Adds non-linearity, enabling the model to capture complex relationships.
4. Learning rate
   Controls how fast weights update during training. High rates may skip optimal points; too low rates slow convergence.
5. Batch size
   Number of samples processed before updating weights. Small batches offer noisy updates (regularization effect), large batches stabilize learning.
6. Epochs
   Number of complete passes through the training dataset; more epochs help learning but risk overfitting.
7. Dropout rate
   Regularization method—randomly deactivates neurons during training to prevent reliance on specific neurons, reducing overfitting.
8. Optimizer (Adam, RMSprop, SGD)

Algorithm for updating weights efficiently during training—affects speed and quality of learning.

### 3.2.1. Base Model

Features: Lag_1, Lag_2, MA_5, STD_5, MA_20, STD_20 are the features used to predict the closing price.

### 3.2.2. Training model

- Normalization method: MinMaxScaler
  Scales each feature into [0, 1] so that no single variable dominates gradient updates, speeding up convergence and stabilizing training.
- Hidden layer 1 units: 128
  Provides capacity to learn high-level nonlinear interactions; too many units increase model complexity and overfitting risk.
- Activation (hidden layers): ReLU
  Introduces non-linearity, prevents vanishing gradients, and accelerates training by zeroing out negative inputs.
- Dropout rate: 0.2
  Randomly zeroes 20 % of activations during training to force the model to build redundant representations, reducing overfitting.
- Hidden layer 2 units: 64
  Gradually funnels learned features into a lower dimension, refining the representation.
- Hidden layer 3 units: 32
  Further condenses information into a compact feature set before the final prediction.
- Output layer: Dense(1) with linear activation
  Yields a single continuous value for the predicted VND–USD rate, appropriate for regression.
- Loss function: mean_squared_error
  Penalizes larger errors more heavily by squaring the residuals, driving the model toward accuracy.
- Optimizer: Adam(learning_rate=0.001)
  Combines adaptive learning rates and momentum to converge quickly and reliably on complex loss surfaces.
- Batch size: 16
  Balances the noisiness of gradient estimates with training speed.

- Epochs: 50
  Defines the total passes over the training data—enough for convergence without early stopping in the base setup.

Evaluation metrics:

**RMSE (Root Mean Squared Error):**
Measures the square root of the average squared differences between predictions and true values, giving error in the same units as the target. Lower RMSE indicates more accurate predictions.

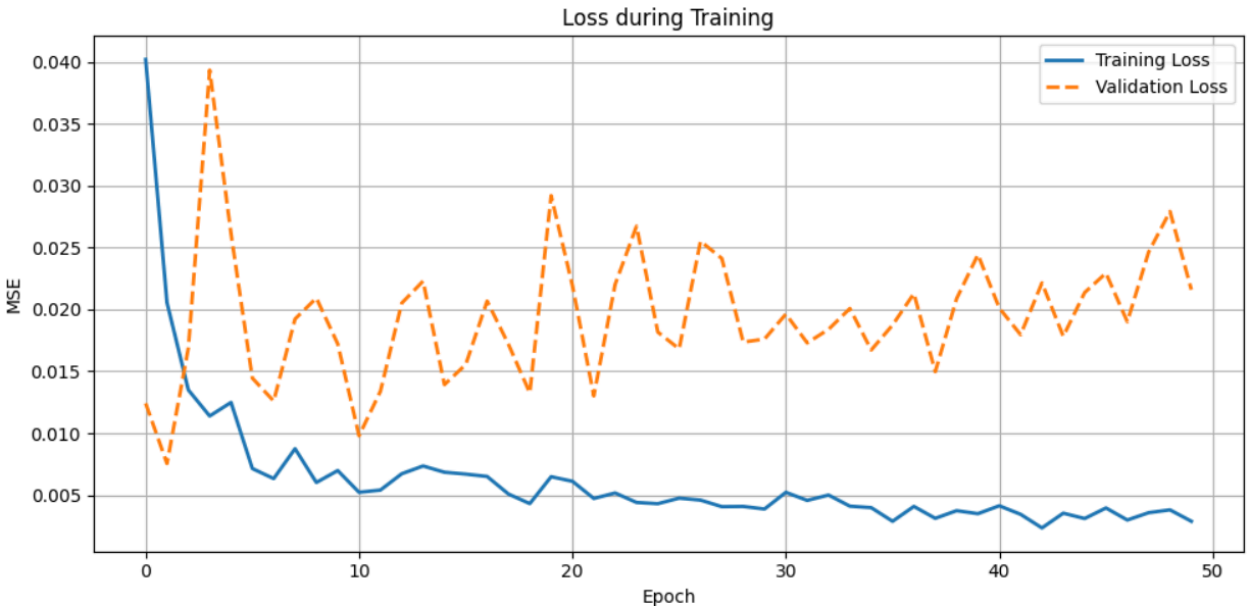
**$R^2$ (Coefficient of Determination):**
Represents the proportion of variance in the target that the model explains. $R^2 = 1$ means perfect fit; $R^2 = 0$ means no better than predicting the mean.

**MAE (Mean Absolute Error):**
Calculates the average absolute difference between predictions and actual values. Because it doesn't square errors, MAE is less influenced by large outliers.

### 3.2.3. Analysis

In this initial run, the model achieves a training RMSE of 42.55 ($R^2 = 0.879$) but a test RMSE of 137.35 ($R^2 = 0.294$). Such a wide gap confirms overfitting: the network memorizes in-sample patterns yet fails to generalize. Notably, validation loss reaches its minimum at epoch 2 (0.0075) before climbing, suggesting that stopping at or near epoch 2–3 would capture the best generalization.

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 54ms/step - loss: 0.0486 - val_loss: 0.0124
Epoch 2/50
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0179 - val_loss: 0.0075
Epoch 3/50
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 0.0153 - val_loss: 0.0169
```

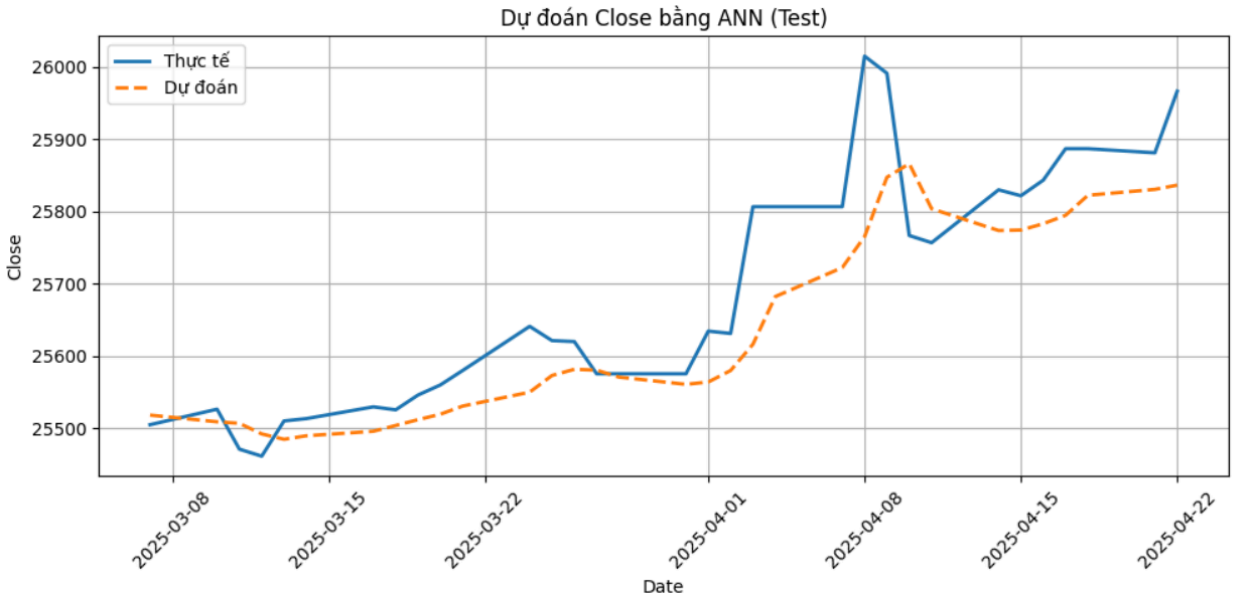



The loss curves demonstrate that the network achieves its best generalization at epoch 2—after which validation loss steadily worsens even as training loss declines, a clear sign of overfitting. To capture that optimal point, it's essential to introduce EarlyStopping (monitoring validation loss with low patience) so that the model retains its early predictive power without memorizing noise.

### 3.2.4. Improve with EarlyStopping

With EarlyStopping, training halted at epoch 3—exactly at the validation‑loss minimum—yielding a train RMSE of 83.07 ($R^2 = 0.540$) and a test RMSE of 82.65 ($R^2 = 0.744$). The near-parity of RMSE values and substantial rise in test $R^2$ (from ~0.29 to 0.74) confirm that the model now generalizes far better; the deliberately "under-trained" in-sample fit (lower $R^2$_train) reflects a trade-off that prevents overfitting and boosts out-of-sample accuracy.

```
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
Test RMSE: 82.65
Test R²  : 0.744
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
Train RMSE: 83.07
Train R²  : 0.540
```




### 3.3 Long Short-Term Memory (LSTM)

Model Overview:

LSTM is a type of recurrent neural network (RNN) that is very powerful in processing and predicting time series data.

Normalization:

Prior to feeding data into the network, we apply MinMax scaling to each feature so that all values lie between 0 and 1, thereby preventing any single predictor's magnitude from disproportionately influencing gradient updates and ensuring faster, more stable convergence.

#### 3.3.1. Base Model

Important Parameters

1. Data and Normalization:

Features: Lag_1, Lag_2, MA_5, STD_5, MA_20, STD_20 are the features used to predict the closing price.

Target: "Close" Closing price (target for the model).

Normalization:

The data is normalized with MinMaxScaler to bring it to the range of 0 to 1.

- scaler_X: Normalize the input features.
- scaler_y: Normalize the target value.

2. LSTM model:

Number of LSTM units (units): 32: This is the number of units of the LSTM layer. Increasing the number of units can help the model learn more complex features, but it can also easily lead to overfitting if it is too large.

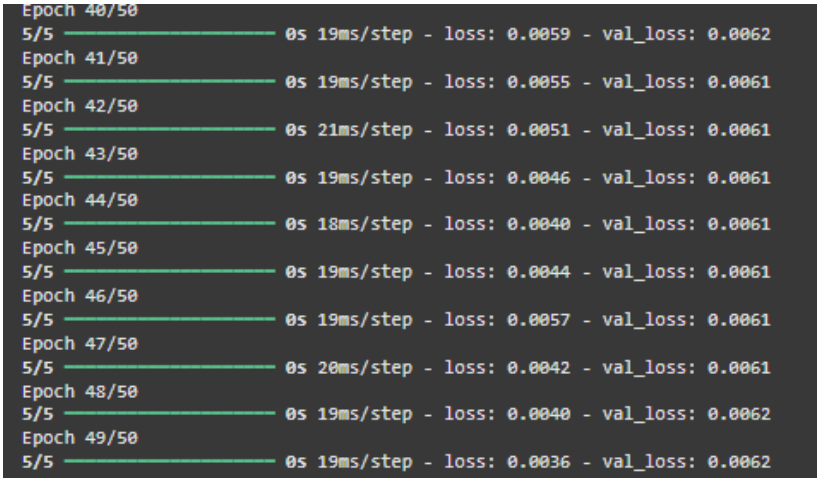
Activation Function: ReLu: Used in the LSTM layer to increase non-linearity and help the model learn better.
Dense Layer (Output Layer): A Dense(1) output layer to predict a unique value for each sample.
Loss Function: mean_squared_error
Optimizer: Adam(learning_rate=0.001)

3. Training Parameters:
Epochs:50: The model is trained for 50 epochs. The number of epochs can be adjusted so that the model learns enough without overfitting.
Batch Size: 16: The number of samples processed in each update of the model's weights. Small batch sizes can help the model learn faster, but they can also increase the variability of the training process.
Validation Data: The model is trained with validation data to monitor performance throughout the training process.

4. Evaluation Metrics:
Mean Squared Error (MSE):
Root Mean Squared Error (RMSE):
Absolute Error (MAE):
MAE is the average absolute value of the deviation between the actual value and the prediction.
R2 (Coefficient of Determination): A measure of how well the model fits the data. An R2 value close to 1 means that the model explains most of the variation in the data.

**3.3.2 Analysis**
1. Loss and val_loss
- Loss drops from 0.1269 to 0.0034, which shows that the model is learning well from the training data.
- Val_loss starts at a high of 0.3952 and drops to 0.0063 towards the end of training, but there are some plateaus.

For example, at epochs 39-50:



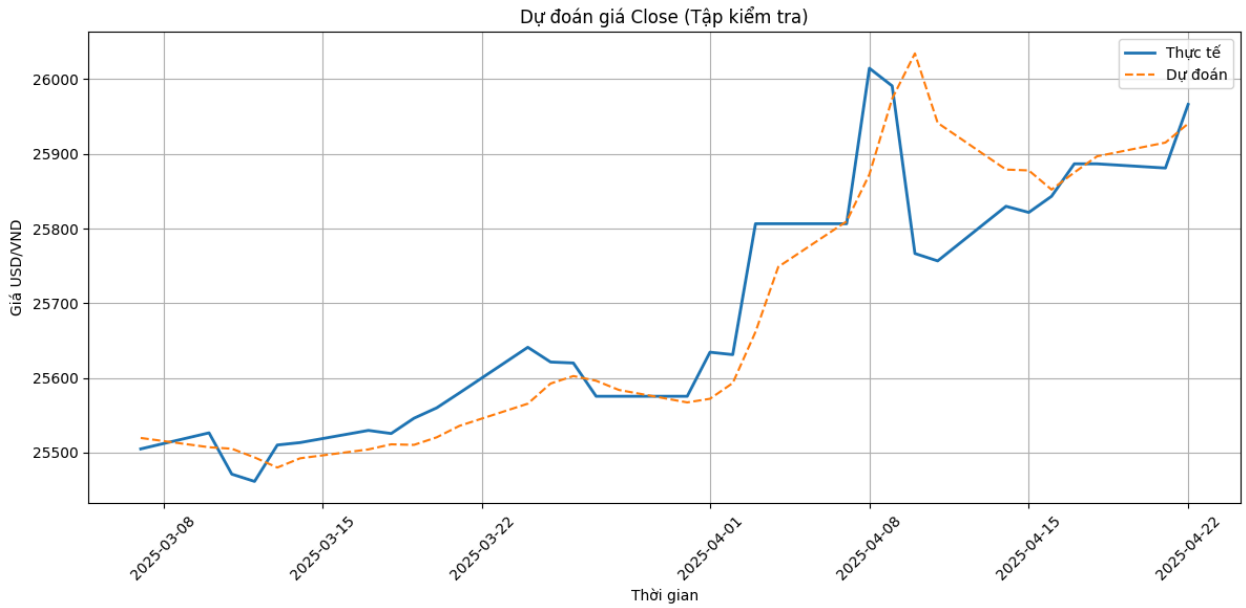

Validation loss does not drop as sharply as the training loss, instead it fluctuates around 0.0061 and 0.0062. This could be a sign of mild overfitting, where the model learns too much on the training set but fails to improve on the test set.

2. Model evaluation results

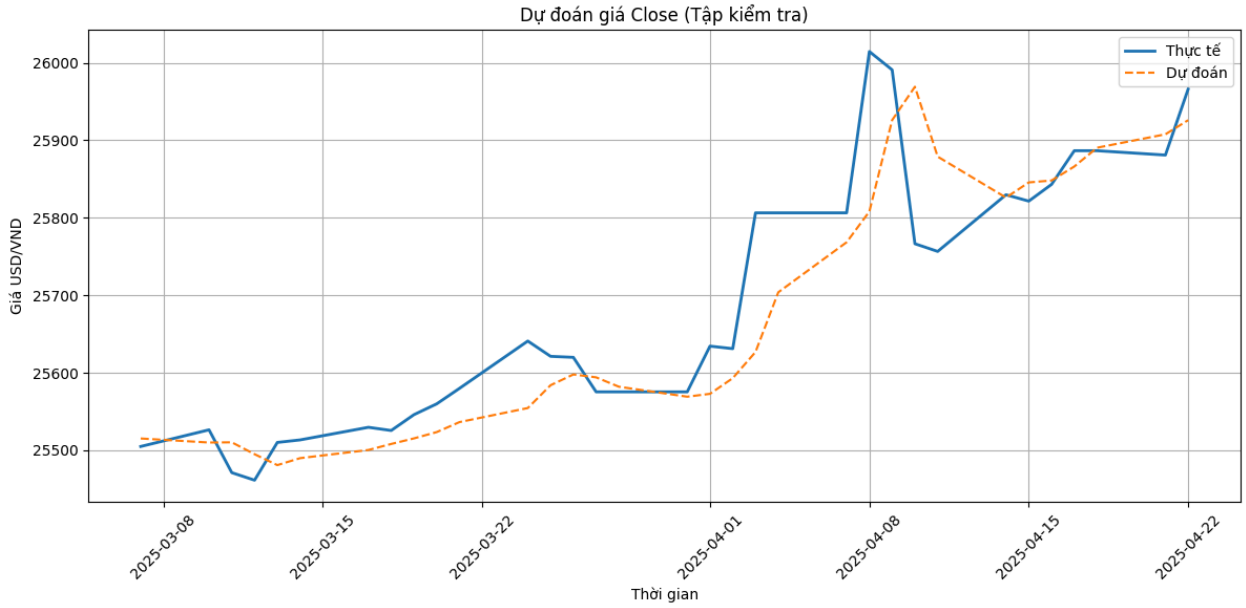
```
Final Evaluation:
MSE: 5479.439080333579
RMSE: 74.02323338205093
MAE: 47.96711441287874
R2: 0.7949463723876351
```

MSE (Mean Squared Error): 5479.439. A fairly high value, but not enough to judge whether the model is really good or not.
RMSE (Root Mean Squared Error): 74.023. This RMSE shows the standard deviation of the prediction, and while not too high, there is still room for improvement.
MAE (Mean Absolute Error): 47.976. This is an acceptable value, but it could be better if the model were further optimized.
R2 (R-squared): 0.794. This is a pretty good metric (ideally close to 1.0), indicating that the model explains about 79% of the variation in the data.

Conclusion: Slight overfitting is suspected, as the model has a plateau in decreasing val_loss and may be learning too closely to the training data. However, this degree of overfitting is not too severe and the model still has good generalization ability.




### 3.3.3 Improve with Early Stopping




```
# 10. Cài đặt EarlyStopping (chỉnh lại)
early_stopping = EarlyStopping(monitor='val_loss', patience=20,
min_delta=0.001, restore_best_weights=True)

# 11. Huấn luyện (với validation data và epoch)
history = model.fit(X_train, y_train, epochs=150, batch_size=batch_size,
validation_data=(X_test, y_test),
                    verbose=1, callbacks=[early_stopping])
```

```
Final Evaluation:
```

```
MSE: 5306.894789445615
RMSE: 72.84843711052156
MAE: 49.28372401515144
R2: 0.8014033896574075
Best Epoch: 45 - loss: 0.0039 - val_loss: 0.0061
```

After going through Early stopping, we see that the model has more opportunities to learn. Some indicators improve such as:

- MSE, RMSE decrease ⇒ the prediction is generally a little more accurate.
- R2 increases to 0.801 ⇒ the model explains more.

However, the model does not learn more after epoch 45 because val_loss does not improve, still stops at 0.61, while the loss only stops at 0.39, which proves that despite the additional epochs (from 50 to 150), the model still has the problem of overfitting.

**4. Fine-Tuning the Models**

In this section, you describe in detail how you tune the parameters (hyperparameters) for each selected model.

**4.1 SVR**

Hyperparameter tuning for the SVR model is performed using a technique called grid search with cross-validation (GridSearchCV). This method systematically tries different combinations of specified hyperparameters to find the best-performing set, based on a chosen evaluation metric.

The tuning process begins by defining a parameter grid that lists several potential values for key SVR hyperparameters: "C", "gamma", and "epsilon". In this project, the model tests 48 combinations of these parameters.

- Four values for C: 1, 10, 100, and 500
- Four values for gamma: 0.01, 0.1, 0.5, and scale
- Three values for epsilon: 0.01, 0.1, and 0.5

These hyperparameters are embedded into the Pipeline, which in this case contains only the SVR model. GridSearchCV then evaluates all possible combinations of these hyperparameters using 5-fold cross-validation. This means the training set is split into five parts; in each iteration, four parts are used to train the model, and the remaining part is used to validate it. The average performance across these folds is used to determine how well each hyperparameter combination performs.

### 4.2 ANN

To hone in on the optimal ANN configuration, I began by fixing a global seed for Python, NumPy, and TensorFlow to ensure reproducible weight initializations and training order. After scaling my six engineered features and splitting the dataset into a 70/30 time-series train/test split, I defined a grid spanning three hidden-layer architectures (64-32, 128-64-32, 256-128-64-32), two activation functions (ReLU, tanh), five learning rates (from 1e-2 down to 5e-5), four batch sizes (8, 16, 32, 64) and five dropout rates (0, 0.1, 0.2, 0.3, 0.4).

```
# 5) Hyperparameter grid
hidden_layers  = [(64,32),(128,64,32),(256,128,64,32)]
activations    = ['relu','tanh']
learning_rates = [1e-2, 1e-3, 5e-4, 1e-4, 5e-5]
batch_sizes    = [8,16,32,64]
dropouts       = [0,0.1,0.2,0.3,0.4]
```

For each combination, the model was compiled and trained for 50 epochs on the training block, then evaluated on both train and test sets. I enforced two validation rules: the test $R^2$ must not exceed the training $R^2$ (no underfitting masquerading as over-generalization), and the absolute $R^2$ difference between train and test must be $\leq 0.1$ (to limit overfitting). Any configuration meeting these criteria was considered valid, and among them I selected the one with the highest test $R^2$.

```
# Train
model.fit(X_tr_s, y_tr_s, epochs=50, batch_size=bs, verbose=0)

# Evaluate Train
y_pred_tr = scaler_y.inverse_transform(model.predict(X_tr_s))
r2_tr = r2_score(y_tr, y_pred_tr)

# Evaluate Test
y_pred_te = scaler_y.inverse_transform(model.predict(X_te_s))
r2_te = r2_score(y_te, y_pred_te)

# Valid if test <= train and diff <= 0.1
diff = abs(r2_tr - r2_te)
valid = (r2_te <= r2_tr) and (diff <= 0.1)
```

In summary, a systematic grid search over architectures, activations, learning rates, batch sizes, and dropout rates—anchored by a fixed random seed and a strict 70/30 time-series split—revealed that a three-layer ReLU network (128-64-32 neurons) trained with a 0.01 learning rate, batch size of 16, and 20 % dropout delivers the best balance of fit and generalization. With an in-sample $R^2$ of 0.866, out-of-sample $R^2$ of 0.803, and an $R^2$ gap

below 0.1, this configuration ensures strong predictive accuracy on unseen data without overfitting.

```
Best configuration:
{'hidden_layers': (128, 64, 32), 'activation': 'relu', 'learning_rate': 0.01, 'batch_size': 16, 'dropout': 0.2,
```

### 4.3 LSTM

In this project, to optimize the performance of the LSTM model, I conduct Grid Search to find the best combination of parameters, including:

- Number of LSTM units
- Learning rate
- Batch size
- Activation function
- Number of LSTM hidden layers

Grid Search is implemented by sequentially trying all possible parameter combinations in the search space.

```
param_grid = {
    'units': [16, 32, 64],
    'learning_rate': [0.001, 0.005],
    'batch_size': [16, 32],
    'activation': ['relu', 'tanh', 'sigmoid'],
    'n_hidden_layers': [1, 2, 3]  # số hidden layers
```

The result:

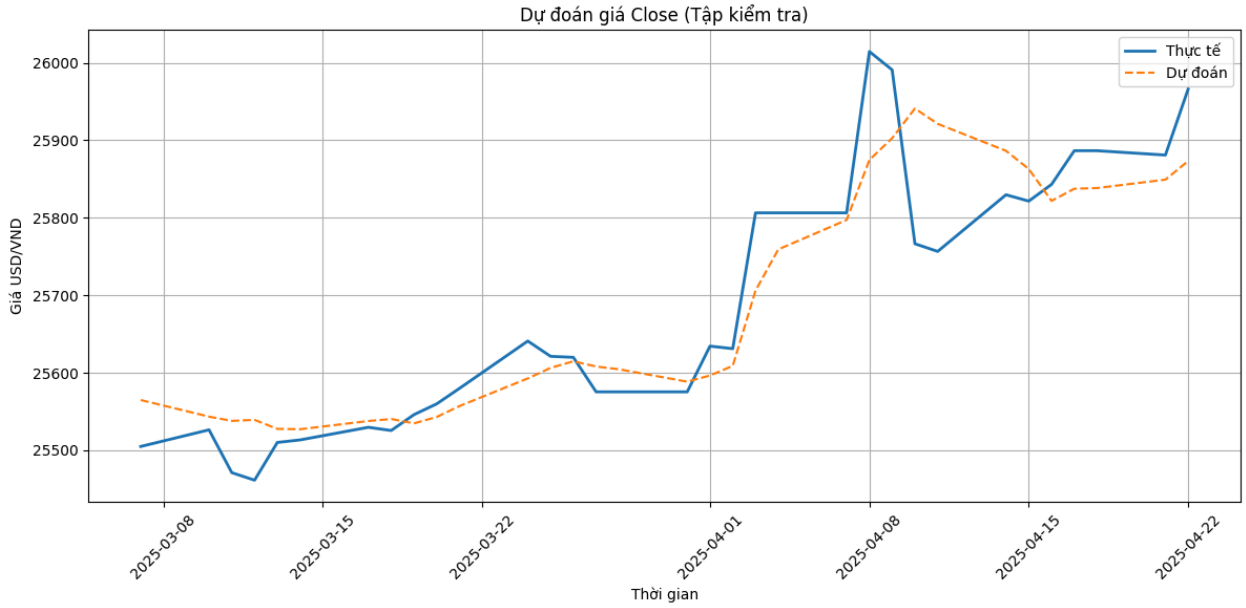
```
Best Parameters:
units: 32
learning_rate: 0.005
batch_size: 32
activation: tanh
n_hidden_layers: 1
Best MSE: 4233.25900222359

Final Evaluation:
RMSE: 65.06349976925304
MAE: 48.33512804924237
R2: 0.8415813913974928
```

Compared with the pre-fine-tuning model:

MSE and RMSE decreased: MSE decreased from 5306.89 to 4233.26 and RMSE decreased from 72.85 to 65.06. This shows that the post-fine-tuning model improved its predictive ability, with a smaller average prediction error.

MAE decreased slightly: MAE decreased from 49.28 to 48.34, although the change was not very large, it showed that the post-fine-tuning model reduced the absolute errors slightly.
R2 increased: R2 increased from 0.8014 to 0.8416, showing that the post-fine-tuning model improved its ability to explain the variance of the data. The 0.04 increase in R2 shows that the model became more effective in capturing the relationship between the variables.




**5. Comparison and Results**

In this section, you will present the evaluation results of the models after fine-tuning and compare their performance. We can compare the performance of the models as follows:

1. SVR:

MSE = 5542.15

RMSE = 74.45

MAE = 54.08

R2 ≈ 0.79

2. ANN:

No direct MSE, RMSE, MAE, but R2 (test) = 0.803

3. LSTM:

MSE = 4233.26

RMSE = 65.06

MAE = 48.34

R2 = 0.8416

The LSTM model shows the best performance in all aspects:

- Has the highest R2 (0.8416), indicating the best ability to explain the variance of the data.
- Has the lowest MSE, RMSE and MAE, indicating the lowest prediction error.

The ANN model has a relatively good R2 (0.803) but it's lower than the LSTM. The SVR model has higher error metrics and lower R2 than the LSTM, suggesting worse performance.

**6. Conclusion**

In conclusion, among the models considered for forecasting the USD/VND exchange rate, the Long-Term Prediction Neural Network (LSTM) model shows superior performance. This model not only effectively captures the complex fluctuations of the exchange rate but also provides accurate forecasts with minimal errors, as shown by the lowest MSE, RMSE, and MAE indices, along with the highest coefficient of determination R2.

This study demonstrates the great potential of deep learning models in forecasting exchange rates. However, it should be noted that forecasting exchange rates is a complex task, influenced by many unpredictable factors. Therefore, the forecast results need to be carefully considered and combined with macroeconomic analysis and other market factors to make the final decision.